

PROJECT IMPLEMENTATION GUIDE

Smart Parking System

Breadboarding and Testing Guide

Step-by-step instructions for assembling the prototype on a breadboard, testing each subsystem, and integrating everything into a working unit.

Estimated time:

4-6 hours for full assembly and testing

Recommended team size: 2-3 people

Before You Start

Required Tools

- Soldering iron (for any component that needs leads tinned)
- Wire strippers / pliers
- Multimeter (essential for troubleshooting)
- Computer with Arduino IDE installed
- Phone or laptop for testing the web app

Required Software (install before starting)

1. **Arduino IDE 2.0+** from [arduino.cc](https://www.arduino.cc)
2. **ESP32 board package** — in Arduino IDE: File → Preferences → add this URL to Additional Boards Manager:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

3. Then go to Tools → Board → Boards Manager → search **esp32** → Install
4. **Required libraries** — in Library Manager (Sketch → Include Library → Manage Libraries):
 - LiquidCrystal I2C by Frank de Brabander
 - ESP32Servo by Kevin Harrington
 - Firebase ESP32 Client by Mobizt

Working Strategy

The system has many subsystems. Trying to wire everything at once is the fastest way to get stuck with a broken setup that's impossible to debug. Instead, follow this golden rule:

Build one subsystem at a time. Test it. Make sure it works. Only then move to the next.

This guide is designed exactly that way. Each phase ends with a checkpoint. Do NOT skip checkpoints.

Phase Overview

Phase	What you build	Time
1	Power rails on breadboard	10 min
2	ESP32 + LCD (display test)	30 min
3	Servo motor (gate test)	20 min
4	Ultrasonic sensors with voltage dividers	30 min
5	First 74HC595 (LED outputs)	45 min

Phase	What you build	Time
6	First 74HC165 (switch input)	30 min
7	Daisy-chain second 74HC595 and 74HC165	30 min
8	WiFi and Firebase connection	30 min
9	Full system integration	45 min
10	Web app testing and live scalability demo	30 min

Phase 1 — Set Up Breadboard Power Rails

Every breadboard has two long rails on each side: one for positive voltage, one for ground. Connect them properly first so every other phase has clean power to draw from.

Steps

5. Place your breadboard with the long rails clearly visible (red and blue stripes).
6. On a **standard 830-point breadboard**, the four rails are split into two halves down the middle. Use jumper wires to bridge them.
7. Bridge the **two top red rails** together (left half to right half).
8. Bridge the **two top blue rails** together.
9. Bridge **top red to bottom red**, and **top blue to bottom blue**, so all power and ground rails are connected.

WHY: On most breadboards, the rails are NOT continuous across the middle. Skipping this step causes mysterious power problems halfway through the project.

Naming convention for this guide

- **RED rail = +5V** (comes from ESP32 5V pin or external supply)
- **BLACK/BLUE rail = GND** (common ground for everything)
- We will add a **+3.3V rail** later — use a separate row in the middle of the breadboard for this, since 3.3V is only used by the shift register chips.

Checkpoint 1

- ☒ All four power rails are bridged with jumpers
- ☒ Visually verify red and blue stripes are connected end-to-end

Phase 2 — ESP32 and LCD

Get the ESP32 connected and the LCD displaying text. This is the simplest subsystem and a great way to confirm your computer can flash firmware to the board.

Steps

10. Place the **ESP32 board** across the middle gap of the breadboard so its pins are split between the two halves. The USB port should hang off one end.
11. Connect **ESP32 5V** → **+5V red rail** using a jumper.
12. Connect **ESP32 GND** → **GND blue rail**.
13. Connect **ESP32 3.3V** → **a free middle row** — this is your +3.3V rail. Use a different color jumper (orange or yellow) so you can tell it apart from +5V.
14. Connect the LCD's I2C backpack:
 -  • VCC → +5V rail
 -  • GND → GND rail
 -  • SDA → ESP32 GPIO 21
 -  • SCL → ESP32 GPIO 22

First firmware test

15. Open Arduino IDE.
16. Open the SmartParking.ino file.
17. Tools → Board → **ESP32 Dev Module**.
18. Tools → Port → select your ESP32's COM port (Windows) or /dev/cu.SLAB_USBtoUART (Mac).
19. At the top of the code, change USE_FIREBASE to false temporarily — we don't want WiFi yet.
20. Click **Upload** (the right-arrow button).
21. Open **Serial Monitor** (the magnifying glass icon, top right). Set baud rate to 115200.

Expected result

On the Serial Monitor you should see:

```
=== Smart Parking Booting ===
[INIT] sensors and servo ready
[WIFI] offline
=== System Ready ===
```

On the LCD, after a moment, you should see:

```
Available Slots:
4 / 4
```

Troubleshooting

LCD shows nothing or garbage: Check SDA/SCL aren't swapped. Adjust the contrast pot on the back of the LCD with a small screwdriver. The I2C address might not be 0x27 — try 0x3F by editing the line `LiquidCrystal_I2C lcd(0x27, 16, 2)` in the code.

Upload fails: Hold the BOOT button on the ESP32 while clicking Upload. Release after "Connecting..." appears in the log.

Checkpoint 2

- ☒ Firmware compiles and uploads without errors
- ☒ Serial Monitor shows 'System Ready' message
- ☒ LCD displays 'Available Slots: 4 / 4'

Phase 3 — Servo Motor (Gate Test)

Wire up the servo so we can verify the gate mechanism before adding sensors.

DO NOT connect the servo's red wire to the ESP32's 5V pin. The ESP32 cannot supply enough current — it will brown-out and reset every time the servo moves. The servo MUST be powered from your external 5V/2A adapter.

Steps

22. Plug your **external 5V adapter** into a separate +5V rail (use a free row, not the same one as ESP32 5V).

23. Connect the external supply's **GND** to the same GND rail as the ESP32. They **MUST** share ground.

24. Wire the servo:

-  • **Red wire (VCC)** → external 5V rail
-  • **Brown/black wire (GND)** → GND rail
-  • **Orange/yellow wire (signal)** → ESP32 GPIO 26

25. Place a **470µF electrolytic capacitor** between the external 5V rail and GND, as close as possible to the servo. This handles the current spike when the servo starts moving.

CHECK CAPACITOR POLARITY: electrolytic capacitors have a + and - side. The longer leg is +. Most capacitors also have a stripe pointing to the - leg. + goes to 5V, - goes to GND. Connecting backwards can make it explode.

Test

Reset the ESP32 (press the EN button). The setup function moves the servo to closed position briefly. You should see the servo arm move once at boot, then stop.

If you have time, you can write a tiny test sketch that opens and closes the servo to verify the range:

```
#include <ESP32Servo.h>
Servo s;
void setup() {
  s.attach(26);
  s.write(0);   delay(1000);
  s.write(90);  delay(1000);
  s.write(0);
}
void loop() {}
```

Troubleshooting

Servo doesn't move at all: Verify the external 5V supply is actually delivering 5V (use multimeter). Check signal wire isn't loose. Try a different GPIO pin temporarily (the ESP32Servo library supports many pins).

Servo twitches randomly: You probably forgot to share grounds, or the bulk capacitor is missing/wrong polarity.

ESP32 keeps resetting: Servo is drawing power from the ESP32 by mistake. Disconnect everything and re-verify the servo's red wire goes to the EXTERNAL supply, not ESP32.

Checkpoint 3

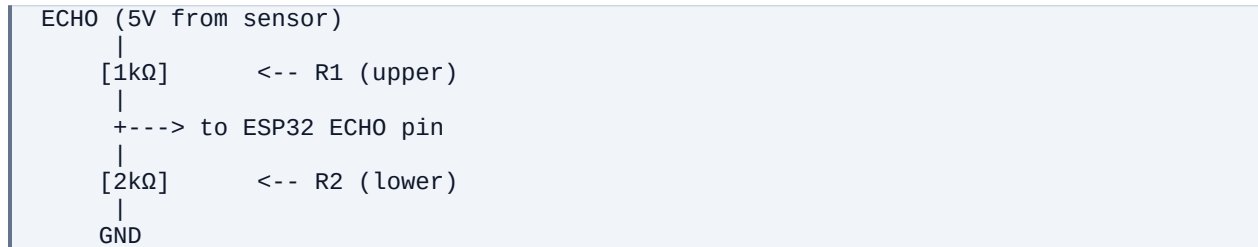
- ☒ Servo moves at boot without ESP32 resetting
- ☒ External supply is providing 5V (verified with multimeter)
- ☒ Servo holds its position (not buzzing or twitching when idle)

Phase 4 — Ultrasonic Sensors

Add both HC-SR04 sensors with their voltage dividers. The voltage divider is essential — the sensor's ECHO pin outputs 5V which can damage the ESP32's 3.3V inputs over time.

Voltage divider explanation

A voltage divider uses two resistors to safely drop the sensor's 5V echo signal to about 3.3V before it reaches the ESP32:



Wiring sensor 1 (US1)

- 26. VCC → +5V rail
- 27. GND → GND rail
- 28. TRIG → ESP32 GPIO 13 (direct connection, no divider)
- 29. ECHO → through voltage divider → ESP32 GPIO 12:
 - ECHO pin → **1kΩ resistor** → junction point
 - Junction point → ESP32 GPIO 12
 - Junction point → **2kΩ resistor** → GND

Wiring sensor 2 (US2)

Same exact pattern but with different ESP32 pins:

- VCC → +5V
- GND → GND
- TRIG → GPIO 14
- ECHO → through 1kΩ/2kΩ divider → GPIO 27

Test

Reset the ESP32 and watch the Serial Monitor. Wave your hand close (within 15cm) of US1, then move it away. You should see:

```

[US1] CAR detected
[US1] clear
  
```

Repeat for US2 — same pattern with [US2] in the message.

If both sensors detect properly, the gate FSM will activate. If you trigger US1 alone for 1.5 seconds, the servo should open. Cover US2 next, the gate eventually closes. Same in reverse for US2 → US1 (an exit).

Troubleshooting

Sensor never reports CAR: Check that VCC is actually 5V (not 3.3V — sensor needs 5V). Verify TRIG and ECHO aren't swapped. Check voltage divider with a multimeter — when ECHO fires, the junction should briefly read ~3.3V.

Sensor reports CAR all the time: Something is in front of it. Move objects away. If still constant, the sensor may be wired wrong or defective — try the other sensor in the same spot.

Wild fluctuating readings: The sensor needs steady 5V. If powered through long thin wires the voltage drops. Use thick power leads to the sensor.

Checkpoint 4

- ☒ US1 reports 'CAR detected' / 'clear' as you wave your hand
- ☒ US2 also responds correctly
- ☒ Voltage divider checks out (~3.3V on ESP32 ECHO pin during pulse)
- ☒ Servo opens/closes when you simulate a car at the gate

Phase 5 — First 74HC595 (LED Outputs)

Time to add LED control via shift register. Get one chip working first, then we'll daisy-chain the second one in Phase 7.

Pin map for 74HC595

Pin	Name	Connect to
1	Q1 (output)	Slot 1 RED LED via 220Ω
2	Q2 (output)	Slot 1 GREEN LED via 220Ω
3	Q3 (output)	Slot 2 RED LED via 220Ω
4	Q4 (output)	Slot 2 GREEN LED via 220Ω
5-7	Q5-Q7	Leave open (phantom slots)
8	GND	GND rail
9	Q7'	(later, to second chip's pin 14)
10	MR / RST $\bar{}$	+3.3V rail (disables reset)
11	SHCP / SCLK	ESP32 GPIO 33
12	STCP / RCLK	ESP32 GPIO 25
13	OE	GND (enables outputs)
14	DS / SER	ESP32 GPIO 32
15	Q0	(unused first bit — leave open)
16	VCC	+3.3V rail

CRITICAL: Pin 10 must connect to +3.3V. Pin 13 must connect to GND. If you forget either one, the chip won't output anything and you'll spend hours debugging.

WHY THE BIT MAPPING IS CONFUSING: The first slot's red LED actually goes to Q0 according to our code (bit 0 = pin 15). But our code uses bit ($i*2$) for slot's red LED. So the actual pin layout for slot 1 is Q0 (pin 15) for RED and Q1 (pin 1) for GREEN. We chose to skip Q0 and use Q1-Q4 for slots 1-2 to make the daisy chain easier to lay out. Read this carefully — it's the most common mistake.

Corrected pin mapping (matching our firmware)

Our firmware uses these bit positions per slot:

- **Slot 1 RED** = bit 0 = Q0 (pin 15)
- **Slot 1 GREEN** = bit 1 = Q1 (pin 1)
- **Slot 2 RED** = bit 2 = Q2 (pin 2)
- **Slot 2 GREEN** = bit 3 = Q3 (pin 3)

- Bits 4-7 = Q4-Q7 (pins 4-7) → phantom slots 3 and 4, leave unconnected

Wiring an LED with current-limiting resistor

For each LED:

30. Take a **220Ω resistor** and one leg of the LED.
31. Connect: shift register output pin → 220Ω resistor → **LED anode (longer leg)**.
32. LED cathode (shorter leg) → GND rail.

LED POLARITY MATTERS: The longer leg is + (anode), the shorter leg is - (cathode). Look inside the LED — the smaller piece of metal is the anode. If wired backwards, the LED simply won't light.

Wiring summary for slots 1 and 2

After this phase you should have:

- Q0 (pin 15) → 220Ω → Slot 1 Red LED → GND
- Q1 (pin 1) → 220Ω → Slot 1 Green LED → GND
- Q2 (pin 2) → 220Ω → Slot 2 Red LED → GND
- Q3 (pin 3) → 220Ω → Slot 2 Green LED → GND
- Q4-Q7 left open for now

Decoupling capacitor

Place a **100nF ceramic capacitor** between the chip's VCC (pin 16) and GND (pin 8), as close as possible to the chip. This filters out tiny voltage noise that would otherwise corrupt data.

Test

Reset the ESP32. At boot, the firmware sends `updateAllLEDs()` which should turn all green LEDs ON for slots 1 and 2 (since they start as available).

Expected result:

- Slot 1 GREEN LED: ON
- Slot 2 GREEN LED: ON
- All other LEDs: OFF

Troubleshooting

All LEDs off: Check pin 13 (OE) is GND. Check pin 10 (MR) is +3.3V. These are the most common mistakes.

Wrong LEDs lit: Bit-to-pin mapping is wrong. Re-check that Q0 = pin 15 (NOT pin 1) — this trips up everyone.

LEDs flicker rapidly: Loose breadboard connection on DATA, CLOCK, or LATCH. Press them down firmly.

LEDs are dim or wrong color: Resistor too high, or LED polarity reversed.

Checkpoint 5

- ☒ Slots 1 & 2 GREEN LEDs are lit at boot
- ☒ No flickering or random behavior
- ☒ Decoupling capacitor installed close to chip

Phase 6 — First 74HC165 (Switch Inputs)

Now add switch reading. The 74HC165 reads 8 parallel switch inputs and shifts them out serially to the ESP32.

Pin map for 74HC165

Pin	Name	Connect to
1	PL / LOAD $\bar{D}$	ESP32 GPIO 23
2	CP / CLK	ESP32 GPIO 19
3	E (D4)	Through 10k Ω to +3.3V (no switch — phantom)
4	F (D5)	Through 10k Ω to +3.3V (no switch — phantom)
5	G (D6)	Through 10k Ω to +3.3V (no switch — phantom)
6	H (D7)	Through 10k Ω to +3.3V (no switch — phantom)
7	Q $\bar{H}$	Leave unconnected
8	GND	GND rail
9	QH	ESP32 GPIO 18 (data out)
10	SER	(later, from second chip's pin 9)
11	A (D0)	Slot 1 switch + 10k Ω pullup
12	B (D1)	Slot 2 switch + 10k Ω pullup
13	C (D2)	Through 10k Ω to +3.3V (no switch — phantom)
14	D (D3)	Through 10k Ω to +3.3V (no switch — phantom)
15	CLK INH	GND (enables clock)
16	VCC	+3.3V rail

CRITICAL: Pin 15 (CLK INH) must go to GND. Pin 10 (SER) needs to be connected — for now, since there's no second chip yet, connect it to +3.3V. Pins 11-14 and 3-6 ALL need pullup resistors even on phantom slot inputs, otherwise they float and read unpredictably.

Wiring a switch

For slot 1 and slot 2 (the real slots):

33. One terminal of the switch → **input pin** (A for slot 1, B for slot 2)
34. Other terminal of the switch → **GND rail**
35. From the input pin, also connect a **10k Ω resistor** → **+3.3V** (this is the pullup)

Behavior: when switch is open, the pullup pulls the input to HIGH (3.3V). When switch is closed (pressed), it shorts the input to GND, reading LOW.

Wiring phantom slot inputs

For pins 3, 4, 5, 6, 13, 14 (the unused inputs):

- Connect each pin to +3.3V through its own 10kΩ pullup resistor
- Don't connect any switch — leave them floating with just the pullup

If you skip the pullups on unused inputs, those bits will read randomly LOW (occupied) and the firmware will think phantom slots are taken. The LCD count will be wrong.

Decoupling capacitor

Place a **100nF ceramic capacitor** between pin 16 (VCC) and pin 8 (GND).

Test

Reset the ESP32. With no switches pressed:

- LCD should show '4 / 4'
- All 4 GREEN LEDs (slots 1, 2, 5, 6) should be ON

Now press slot 1's switch:

- LCD changes to '3 / 4' .
- Slot 1 RED LED turns ON, GREEN turns OFF
- Serial Monitor shows '[SWITCH 1] occupied'

Release the switch:

- LCD returns to '4 / 4'
- Slot 1 GREEN LED turns ON, RED turns OFF
- Serial shows '[SWITCH 1] free'

Repeat for slot 2's switch.

Troubleshooting

Slots show 'occupied' even when nothing is pressed: Pin 15 (CLK INH) isn't grounded, OR pin 10 (SER) isn't tied to +3.3V, OR pullup resistors are missing on inputs.

Switch press doesn't register: Verify the switch actually closes when pressed (test with multimeter in continuity mode). Verify the switch wires are crossed correctly — one to GND, one to the input pin.

LCD count is off: Phantom inputs missing pullups — they're reading as 'pressed'. Add 10kΩ resistors to +3.3V on each unused input pin.

Checkpoint 6

- ☒ LCD shows 4 / 4 at boot

- ☒ Pressing slot 1 switch decrements count and turns LED red
- ☐ Pressing slot 2 switch does the same
- ☒ Releasing either switch returns LED to green and increments count

Phase 7 — Daisy-Chain the Second Chips

Now we add the second 74HC595 and the second 74HC165. This is the most important phase — it proves the daisy chain architecture works, which is the whole point of the project's scalability story.

Adding the second 74HC595

Place the second chip on the breadboard. All wiring is the same as the first chip EXCEPT:

36. Chip 2 pin 14 (DS/SER) ← connect to Chip 1 pin 9 (Q7'). This is the daisy-chain link.
37. Chip 2 outputs go to slots 5 and 6:
 - Q0 (pin 15) → 220Ω → Slot 5 Red LED → GND
 - Q1 (pin 1) → 220Ω → Slot 5 Green LED → GND
 - Q2 (pin 2) → 220Ω → Slot 6 Red LED → GND
 - Q3 (pin 3) → 220Ω → Slot 6 Green LED → GND
 - Q4-Q7 left open (phantom slots 7 & 8)
38. All control lines are SHARED with chip 1: connect chip 2 pin 11 (CLOCK) to the same wire as chip 1 pin 11. Same for pin 12 (LATCH).
39. Pin 13 (OE) → GND, Pin 10 (MR) → +3.3V (same as chip 1)
40. 100nF decoupling cap close to the chip.

Adding the second 74HC165

Same idea but for switches:

41. Chip 2's pin 9 (QH) → chip 1's pin 10 (SER). This replaces the previous +3.3V connection on chip 1's pin 10.
42. Chip 2's pin 10 (SER) → +3.3V (since this is now the LAST chip in the chain).
43. Chip 2 inputs go to slots 5 and 6:
 - Pin 11 (A) → Slot 5 switch + 10kΩ pullup
 - Pin 12 (B) → Slot 6 switch + 10kΩ pullup
 - Pins 3-6, 13, 14 → all need 10kΩ pullups to +3.3V (phantom inputs)
44. Shared signals: connect chip 2 pin 1 (LOAD), pin 2 (CLOCK), pin 15 (CLK INH=GND), pin 16 (VCC=3.3V), pin 8 (GND) the same as chip 1.
45. 100nF decoupling cap.

DOUBLE-CHECK: When you add chip 2 to the 165 input chain, you **MUST** disconnect chip 1's pin 10 from +3.3V. The +3.3V now only goes to chip 2's pin 10 (the last chip). If you leave both connected, the data won't shift correctly.

Test

Reset the ESP32. With nothing pressed:

- LCD shows '4 / 4'
- All 4 GREEN LEDs (slots 1, 2, 5, 6) lit

Press each switch in turn:

- Slot 1 switch → slot 1 LED red, count drops to 3
- Slot 2 switch → slot 2 LED red, count drops
- Slot 5 switch → slot 5 LED red, count drops
- Slot 6 switch → slot 6 LED red, count drops

All four working independently means the daisy chain is functional.

Troubleshooting

Slots 1 & 2 work but slots 5 & 6 don't: Chip 2 isn't getting data. Check the daisy-chain link from chip 1's pin 9 (Q7') to chip 2's pin 14 (DS). Verify chip 2's CLOCK and LATCH share the same nets as chip 1.

Slots 5 & 6 work but slots 1 & 2 don't: You may have swapped the chip order. Chip 1 should connect directly to ESP32. Chip 2 receives from chip 1's overflow.

All slots show occupied at boot: 165 chain has wrong SER pin connection. Verify chip 1 pin 10 connects to chip 2 pin 9 (NOT to +3.3V), and chip 2 pin 10 connects to +3.3V.

Checkpoint 7

- ☐ All 4 real slots work independently
- ☐ LCD count adjusts correctly
- ☐ LED colors switch correctly per slot
- ☐ Phantom slots 3, 4, 7, 8 are NOT visible (no LEDs lit for them since none are wired)

Phase 8 — WiFi and Firebase

Time to bring the system online. This phase doesn't require any new hardware — just code changes.

Steps

46. In `SmartParking.ino`, change `USE_FIREBASE` back to `true`.

47. Update the WiFi credentials at the top:

```
#define WIFI_SSID      "YourWiFiName"
#define WIFI_PASSWORD  "YourPassword"
```

48. Update the Firebase secret. Get it from *Firebase Console* → *Project Settings* → *Service Accounts* → *Database Secrets* → *Show*:

```
#define FIREBASE_AUTH  "your_database_secret_here"
```

49. Make sure WiFi is **2.4GHz** — ESP32 cannot connect to 5GHz networks. If your home router is dual-band, connect to the 2.4GHz SSID specifically.

50. Upload the firmware.

Expected serial output

```
=== Smart Parking Booting ===
WiFi.....
[WIFI] connected, IP = 192.168.x.x
[STREAM] listening for app changes
[FIREBASE] ready
=== System Ready ===
```

Verify in Firebase

51. Open Firebase Console → Realtime Database.

52. You should see data populated automatically:

```
smartparking-93af3
├── slots/
│   ├── slot1: { occupied: false, reserved: false }
│   ├── slot2: { occupied: false, reserved: false }
│   ├── slot3: { occupied: false, reserved: false } <-- phantom
│   ├── slot4: { occupied: false, reserved: false } <-- phantom
│   ├── slot5: { occupied: false, reserved: false }
│   ├── slot6: { occupied: false, reserved: false }
│   ├── slot7: { occupied: false, reserved: false } <-- phantom
│   └── slot8: { occupied: false, reserved: false } <-- phantom
└── system/
    ├── totalSlots: 4 <-- shows REAL_SLOTS, not NUM_SLOTS
    └── availableSlots: 4
```

Troubleshooting

Stuck on 'WiFi.....': WiFi credentials wrong, or router is 5GHz only, or signal too weak.

WiFi connects but no [FIREBASE] ready: Database secret is wrong. Re-copy it carefully — it's a long string with no spaces.

[STREAM] failed to begin: Database rules might be blocking. In Firebase Console → Realtime Database → Rules, set both .read and .write to true temporarily for testing.

Checkpoint 8

- ☒ Serial Monitor shows '[FIREBASE] ready'
- ☐ Firebase Console shows the slots data tree
- ☒ LCD still works correctly with switches

Phase 9 — Full System Integration Test

All hardware is wired and the firmware connects to the cloud. Now run a full integration test to make sure everything works together.

Test sequence

53. **Boot test:** Reset ESP32. LCD shows '4 / 4', all 4 green LEDs on, Firebase reads '4 available'.
54. **Switch test (slot 1):** Press the limit switch. LCD '3 / 4', LED red, Firebase shows slot1 occupied=true. Release: back to '4 / 4'.
55. **Multi-slot test:** Press all 4 switches simultaneously. LCD shows 'FULL'. Release them one by one and watch LCD count up.
56. **Gate test (entry):** Cover US1 with your hand for 1.5 seconds. The servo should open. Then cover US2 (simulating car passing through). The servo closes after 2 seconds.
57. **Gate test (exit):** Cover US2 first this time. Servo opens. Cover US1 next. Servo closes.
58. **Stability test:** Leave the system idle for 5 minutes. Watch Serial Monitor. There should be NO unexpected state changes — no spurious switches reading occupied, no false gate openings.

Common integration issues

Servo opens randomly at boot: A sensor reading is glitching at startup. Add a 1-second delay between sensor initialization and the gate FSM starting.

LCD updates but Firebase doesn't: Stream not running. Check serial — does it say '[STREAM] listening for app changes'? If not, the stream failed. Restart and watch boot messages.

Some switches work, others don't: Inspect those specific input pins for pullup resistors. Phantom inputs especially love being missed.

Checkpoint 9

- ☐ All 4 real switches work and update Firebase in real time
- ☐ Both gate scenarios (entry, exit) work correctly
- ☐ System runs stably for 5+ minutes with no false events
- ☐ Firebase data tree always matches the physical hardware state

Phase 10 — Web App Testing and Live Demo

The hardware is fully working. Now connect the web app and rehearse the demo.

Open the web app

59. On your phone or laptop, open:

```
https://smartparking-93af3.web.app
```

60. Sign in with the email and password you set up in Firebase Authentication.

Verify dashboard

- App shows 4 slot cards labeled 'Slot 1', 'Slot 2', 'Slot 5', 'Slot 6'
- Phantom slots (3, 4, 7, 8) are NOT visible
- Summary card shows 'Available 4 / of 4 total'
- Connection dot at top right is GREEN

Live integration tests

61. **Switch** → **app**: Press slot 1's limit switch on the breadboard. Within 1-2 seconds, the app's slot 1 card should turn red and show 'Occupied'.
62. **App** → **hardware**: Click 'Reserve' on slot 2 in the app. Within 1-2 seconds, slot 2's RED LED on the breadboard should light up. Click 'Unreserve': LED returns to green.
63. **Reserved + Taken alert**: Reserve slot 5 in the app. Press slot 5's limit switch on the breadboard. Watch the Alerts section in the app — it should pop up 'Reserved slot 5 has been taken.'

Live scalability demo (the killer demo)

This is what proves true scalability to your professor:

64. Start with the 4 slots working as normal.
65. Walk your professor through the system — 'this is slot 1 connected to chip 1 of each shift register, this is slot 5 connected to chip 2.'
66. Now physically connect the spare slot daughterboard to the slot 7 position (chip 2 input C / chip 2 outputs Q4 & Q5).
67. Open Arduino IDE, change `REAL_SLOTS` from 4 to 5.
68. Open `index.html`, change `HIDDEN_SLOTS` from `[3, 4, 7, 8]` to `[3, 4, 8]`.
69. Re-upload firmware and re-run `firebase deploy --only hosting`.
70. Refresh the web app — slot 7 now appears, fully functional!
71. Press the new switch — system recognizes it. The LED responds. Firebase updates in real time.

This single demonstration is the most powerful proof of scalability you can give. The shift register architecture means adding slot 7 took just two small code changes — no rewiring, no pin reassignments, no firmware redesign.

Final checklist before presentation day

- ☐ All 4 slots respond to physical switches and to app reservations
- ☐ Gate opens for both entry and exit scenarios
- ☐ Reservation alerts fire when a reserved slot is taken
- ☐ Web app loads on multiple devices simultaneously (test phone + laptop)
- ☐ Spare slot daughterboard is built and ready for live demo
- ☐ You can confidently explain every line of the firmware

Appendix — General Troubleshooting

My ESP32 won't show up on the COM port

- Try a different USB cable (some cables are charge-only and don't carry data)
- Install or update CP210x or CH340 drivers (depends on your ESP32 variant)
- Try a different USB port

Random resets / brownouts

- Almost always a power issue. Use a thicker external 5V supply.
- Check that the servo isn't drawing from ESP32's 5V
- Add more decoupling capacitors near the ESP32

Code compiles but does nothing

- Did you select the right board? Tools → Board → ESP32 Dev Module
- Did you select the right port? Tools → Port
- Hold the BOOT button while clicking Upload

LCD shows nothing

- Adjust contrast pot on the back of the I2C backpack
- Verify SDA and SCL aren't swapped
- Run an I2C scanner sketch — the LCD might be at 0x3F instead of 0x27

Sensors give noisy readings

- Verify HC-SR04 power is steady 5V (use multimeter under load)
- Make sure voltage divider is wired correctly
- Place sensors on solid surface, not wobbling on jumper wires

Servo behaves erratically

- MUST be powered from external supply, not ESP32
- All grounds must be common
- Bulk capacitor must be installed correctly

Shift register outputs are wrong or missing

- Check pin 13 (OE) is GND
- Check pin 10 (MR/RST) is +3.3V
- Check pin 8 is GND, pin 16 is +3.3V
- Verify daisy-chain link from Q7' to next chip's DS

Shift register inputs always read same value

- Check pin 15 (CLK INH) is GND
- Check pin 1 (LOAD) is connected to ESP32
- Last chip's pin 10 (SER) MUST be +3.3V
- All inputs need 10kΩ pullups to +3.3V (even unused ones)

Firestore data doesn't update

- Check WiFi connection in Serial Monitor
- Verify database secret is correct
- Check Firestore rules — set to permissive for testing: `'.read': true, '.write': true`
- Try refreshing the Firestore Console manually

Web app shows 'Loading...' forever

- Check that Firestore Authentication is enabled
- Verify the email/password you used to log in actually exists in the Auth users list
- Open browser console (F12) and check for errors

If all else fails

- Disconnect EVERYTHING and rebuild the breadboard from Phase 1, testing at each phase
- 90% of mystery problems are caused by loose breadboard connections — push everything down firmly
- Take a photo of your wiring and compare against the diagrams in this guide

Final Notes

This guide gets you from zero to working prototype. Once it works on the breadboard, the PCB just makes it permanent and presentable — but the breadboard version is what proves your design works.

Take photos at each phase. Document any deviations from the guide (different pin choices, etc.) so the team has a record. Most importantly, don't skip checkpoints — passing each one before moving on saves hours of debugging later.

Good luck with the build.